C# / .NET STUDY NOTES

Methods and Parameter Passing in C#

Roadmap Topic 5

Learning focus
Method design, parameters and arguments, return values, pass-by-value, ref/out/in, overloads, tuples, local functions, and recursion.

Examples
Gift validation, fee calculations, allocations, summary methods, reusable business logic, and clear method boundaries.

Supplied study notes → structured sections → examples and flows → revision and interview preparation

Contents

Click a section name to jump directly to that part of the notes.

01 Topic Coverage
02 What Is a Method?
03 Definition
04 Why Do We Need Methods?
05 Benefits of Methods
06 Basic Method Syntax
07 Complete Method Anatomy
08 Calling a Method
09 Method Invocation
10 Method Without Parameters
11 Method with One Parameter
12 Method with Multiple Parameters
13 Parameter vs Argument
14 Parameter
15 Argument
16 Parameter vs Argument Flow
17 Return Type
18 Returning int
19 Returning bool
20 Returning string
21 Returning an Object
22 The return Statement
23 Code After return Does Not Execute
24 Early return
25 void Methods
26 Definition
27 void Can Still Use return
28 When Should a Method Return a Value?
29 When Should a Method Be void?
30 Pass by Value — Default Behavior
31 Value Type Passed by Value
32 Value Type Pass-by-Value Flow
33 Reference Type Passed by Value
34 Reference Type Pass-by-Value Flow
35 Reassigning Reference Parameter
36 Important Pass-by-Value Rule
37 ref Parameter
38 Definition
39 ref Example
40 ref Flow
41 ref Requires Initialization
42 When to Use ref
43 out Parameter
44 Definition
45 out Example
46 out Flow
47 Classic TryParse Example
48 Why TryParse Uses out
49 out Variable Declaration at Call Site
50 out Must Be Assigned
51 ref vs out
52 in Parameter

53 Definition
54 in Example
55 in Flow
56 Why in Exists
57 Do Not Overuse in
58 ref readonly Parameter
59 Parameter Modifier Comparison
60 Important async Restriction
61 Optional Parameters
62 Definition
63 Passing Optional Argument Explicitly
64 Optional Parameter Flow
65 Optional Parameter Rule
66 Optional Parameter Defaults
67 Be Careful with Changing Defaults
68 Named Arguments
69 Positional Arguments
70 Named Arguments Can Change Ordering
71 When Named Arguments Are Useful
72 Named Arguments and API Design
73 params Parameter
74 Definition
75 Calling params with No Arguments
76 Calling params with an Array
77 params Rule
78 Modern params in Current C#
79 params Example — Gift Allocations
80 Method Overloading
81 Definition
82 Why Overloading Exists
83 Overload by Number of Parameters
84 Overload by Parameter Types
85 Return Type Alone Cannot Overload
86 Method Signature
87 Overloading and ref/out/in
88 Bad Overloading
89 Expression-Bodied Methods
90 Expression-Bodied void Method
91 When Expression-Bodied Methods Are Good
92 Static Methods
93 Definition
94 Static Method Flow
95 When Static Methods Are Useful
96 Static Method Limitation
97 Instance Methods
98 Definition
99 Instance Method Flow
100 Static vs Instance Methods
101 Fundraising Example
102 Returning Multiple Values
103 Returning Tuple
104 Tuple Deconstruction
105 Tuple Flow
106 Tuple vs out
107 When Not to Use Tuple
108 Local Functions

109 Definition
110 Why Local Functions Exist
111 Local Function Scope
112 Local Function Capturing Variables
113 Local Function vs Separate Private Method
114 Local Function vs Lambda
115 Recursion
116 Definition
117 Base Case
118 Infinite Recursion
119 When Recursion Is Useful
120 When Not to Use Recursion
121 Method Parameters Are Local Variables
122 Parameter Scope
123 Local Variables Inside Methods
124 Method Call Stack Concept
125 Method Design — One Clear Responsibility
126 Good Method Names
127 Boolean Method Naming
128 Methods Should Not Have Too Many Parameters
129 Parameter Object Pattern Preview
130 Pure Methods
131 Method with Side Effects
132 Fundraising Example — Complete Gift Calculation Met...
133 Fundraising Example — Validate Allocation
134 Fundraising Example — Multiple Outputs
135 Common Mistake — Confusing Argument and Parameter
136 Common Mistake — Expecting Normal Value Parameter t...
137 Common Mistake — Believing Reference Types Are Pass...
138 Common Mistake — Using ref to Return One Value
139 Common Mistake — Too Many out Parameters
140 Common Mistake — Overusing Optional Parameters
141 Common Mistake — Deep Recursion Without Need
142 Common Mistake — Method Doing Too Much
143 Common Mistake — Hidden Side Effects
144 Performance Considerations
145 Large Struct Parameters
146 Avoid Allocations in params Hot Paths
147 Security Considerations
148 Do Not Pass Secrets Unnecessarily
149 Method Boundaries and Validation
150 Practical Exercise — Add Two Values
151 Practical Exercise — Validate Gift
152 Practical Exercise — Optional Fee Rate
153 Practical Exercise — ref
154 Practical Exercise — out
155 Practical Exercise — params
156 Practical Exercise — Tuple
157 Practical Exercise — Recursion
158 Interview Questions
159 What is a method in C#?
160 What is the difference between a parameter and an a...

161 What is a return type?

162 What does void mean?

163 What does return do?

164 Are parameters passed by value or by reference by d...

165 If classes are reference types, are they automatica...

166 Why can a method modify a class object's properties...

167 Why doesn't reassigning the parameter replace the c...

168 What is ref?

169 What is out?

170 What is in?

171 What is the difference between ref, out, and in?

172 Can ref, out, or in be used in async methods?

173 What is an optional parameter?

174 What is a named argument?

175 What is params?

176 What is method overloading?

177 Can methods be overloaded only by changing the retu...

178 What is a method signature?

179 What is an expression-bodied method?

180 What is a local function?

181 What is recursion?

182 What is the difference between static and instance...

183 How can a method return multiple values?

184 Best Practices

185 Quick Revision

186 Complete Fundraising Example

Topic Coverage

A **method** is one of the most important building blocks in C#.

Until now, we learned how to create data, perform calculations, make decisions, and repeat work:

```
Variables
    →
Data Types
    →
Operators
    →
Conditions
    →
Loops
```

But real applications cannot place all logic inside one large block of code.

We divide logic into reusable methods.

For example, instead of writing the same gift validation repeatedly:

```
if (giftAmount > 0m)
{
    // process
}
```

we can create:

```
bool IsValidGift(decimal giftAmount)
{
    return giftAmount > 0m;
}
```

and call:

```
if (IsValidGift(giftAmount))
{
    // process
}
```

This topic covers:

- what a method is;
- why methods exist;
- method declaration;
- method invocation;
- method anatomy;
- return types;
- `void`;
- `return`;
- parameters;

- arguments;
- parameter vs argument;
- passing by value;
- reference-type arguments;
- `ref`;
- `out`;
- `in`;
- `ref readonly`;
- `params`;
- optional parameters;
- default parameter values;
- named arguments;
- positional arguments;
- method overloading;
- method signatures;
- return values;
- returning multiple values;
- tuples;
- local functions;
- expression-bodied methods;
- static methods;
- instance methods;
- recursion;
- method design;
- Fundraising Platform examples;
- performance considerations;
- common mistakes;
- interview questions;
- practical exercises.

The basic flow is:

```
Caller
    →
Pass Arguments
    →
Method Receives Parameters
    →
Method Executes Logic
    →
Returns Result
    →
Caller Continues
```

What Is a Method?

Definition

A **method is a named block of code that performs a specific operation**.

A method can:

- receive input;
- perform calculations;
- validate data;
- modify object state;
- call other methods;
- return a result;
- or perform an action without returning a value.

Example:

```
decimal CalculateRemainingAmount(
    decimal giftAmount,
    decimal allocatedAmount)
{
    return giftAmount - allocatedAmount;
}
```

This method:

```
Receives
    →
Gift Amount
Allocated Amount

Performs
    →
Subtraction

Returns
    →
Remaining Amount
```

Why Do We Need Methods?

Imagine writing this calculation in many places:

```
decimal remaining =
    giftAmount - allocatedAmount;
```

Then later the business rule changes.

Perhaps remaining amount must never become negative.

If this logic exists in:

```
Controller
Service
Background Worker
Import Process
Reporting Code
```

you may need to modify it in many places.

A method gives us one reusable operation:

```
decimal CalculateRemainingAmount(
    decimal giftAmount,
    decimal allocatedAmount)
{
    decimal remaining =
        giftAmount - allocatedAmount;

    return Math.Max(
        remaining,
        0m);
}
```

Now every caller uses the same behavior.

Benefits of Methods

Methods help achieve:

```
Reuse
    +
Readability
    +
Separation of Responsibilities
    +
Testing
    +
Maintainability
    +
Abstraction
```

Compare:

```
if (amount > 0m
    && constituentId > 0
    && !isCancelled
    && paymentSucceeded)
{
    // ...
}
```

with:

```
if (CanProcessGift(
        amount,
        constituentId,
        isCancelled,
        paymentSucceeded))
{
    // ...
}
```

The second version communicates business intention more clearly.

Basic Method Syntax

A normal method can look like:

```
public decimal CalculateGiftFee(
    decimal giftAmount)
{
    return giftAmount * 0.02m;
}
```

Main parts:

```
public
    →
Access Modifier

decimal
    →
Return Type

CalculateGiftFee
    →
Method Name

(decimal giftAmount)
    →
Parameter List

{ ... }
    →
Method Body
```

Complete Method Anatomy

Consider:

```
public static decimal CalculateNetGift(
    decimal grossAmount,
    decimal fee)
{
    decimal netAmount =
        grossAmount - fee;

    return netAmount;
}
```

Breakdown:

```
public
    →
Who can access method

static
    →
Method belongs to type rather than object instance

decimal
    →
Return type

CalculateNetGift
    →
Method name

grossAmount
fee
    →
Parameters

Method body
    →
Business logic

return netAmount
    →
Value returned to caller
```

Calling a Method

Creating a method does not automatically execute it.

You must **call** or **invoke** it.

Example:

```
decimal netAmount =
    CalculateNetGift(
        1000m,
        25m);
```

Flow:

```
Caller
    →
CalculateNetGift(1000, 25)
    →
Method executes
    →
1000 - 25
    →
975
    →
Return 975
    →
netAmount = 975
```

Method Invocation

Another term for calling a method is:

```
Method Invocation
```

Example:

```
ProcessGift();
```

or:

```
decimal total =
    CalculateTotal(
        gifts);
```

Both are method invocations.

Method Without Parameters

A method does not have to receive input.

Example:

```
void DisplayApplicationName()
{
    Console.WriteLine(
        "Fundraising Platform");
}
```

Call:

```
DisplayApplicationName();
```

The empty parentheses mean:

```
No Parameters
```

Method with One Parameter

```
void DisplayGiftAmount(
    decimal giftAmount)
{
    Console.WriteLine(
        giftAmount);
}
```

Call:

```
DisplayGiftAmount(
    1000m);
```

Method with Multiple Parameters

```
decimal CalculateRemaining(
    decimal totalAmount,
    decimal allocatedAmount)
{
    return totalAmount
           - allocatedAmount;
}
```

Call:

```
decimal remaining =
    CalculateRemaining(
        1000m,
        700m);
```

Result:

```
300
```

Parameter vs Argument

This is an important interview distinction.

Consider:

```
decimal CalculateRemaining(
    decimal totalAmount,
    decimal allocatedAmount)
{
    return totalAmount
           - allocatedAmount;
}
```

Here:

```
totalAmount
allocatedAmount
```

are **parameters**.

Now call:

```
CalculateRemaining(
    1000m,
    700m);
```

Here:

```
1000m
700m
```

are **arguments**.

Parameter

A parameter belongs to the method declaration.

```
decimal giftAmount
```

means:

> This method expects a decimal value and refers to it inside the method as `giftAmount`.

Argument

An argument is the actual value supplied by the caller.

Example:

```
ProcessGift(
    1000m);
```

`1000m` is the argument.

Parameter vs Argument Flow

```
Method Declaration

decimal Calculate(decimal giftAmount)
                            ↑
                        Parameter


Method Call

Calculate(1000m)
          ↑
       Argument
```

Return Type

A method's **return type** tells the compiler what kind of value the method produces.

Example:

```
decimal CalculateFee(
    decimal amount)
{
    return amount * 0.02m;
}
```

Return type:

```
decimal
```

The caller can therefore write:

```
decimal fee =
    CalculateFee(
        1000m);
```

Returning int

```
int CountValidGifts(
    decimal[] gifts)
{
    int count = 0;

    foreach (decimal gift in gifts)
    {
        if (gift > 0m)
        {
            count++;
        }
    }

    return count;
}
```

Returning bool

```
bool IsValidGift(
    decimal amount)
{
    return amount > 0m;
}
```

Call:

```
if (IsValidGift(
        giftAmount))
{
    ProcessGift();
}
```

Returning string

```
string GetGiftCategory(
    decimal amount)
{
    return amount switch
    {
        >= 10_000m => "Principal",
        >= 5_000m => "Major",
        >= 1_000m => "Leadership",
        > 0m => "Standard",
        _ => "Invalid"
    };
}
```

Returning an Object

Later, we might have:

```
Gift CreateGift(
    long constituentId,
    decimal totalAmount)
{
    return new Gift(
        constituentId,
        totalAmount);
}
```

Methods can return:

```
Primitive values
Objects
Collections
Tuples
Tasks
Nullable values
Interfaces
Generics
```

and many other types.

The return Statement

`return` ends the current method and optionally sends a result back to the caller.

Example:

```
decimal Add(
    decimal first,
    decimal second)
{
    return first + second;
}
```

When execution reaches:

```
return first + second;
```

the method ends.

Code After return Does Not Execute

This is invalid/unreachable logic:

```
decimal Calculate(
    decimal amount)
{
    return amount * 2;

    // This will never be reached.
    // Console.WriteLine("Done");
}
```

Once `return` executes:

```
Method Ends
```

Early return

`return` can also exit a method early.

Example:

```
bool CanProcessGift(
    decimal amount,
    bool paymentSucceeded)
{
    if (amount <= 0m)
    {
        return false;
    }

    if (!paymentSucceeded)
    {
        return false;
    }

    return true;
}
```

Flow:

```
Amount valid?
    →
No
    →
return false

Yes
    →
Payment succeeded?
    →
No
    →
return false

Yes
    →
return true
```

This is a common **guard clause** style.

void Methods

Definition

A `void` method does not return a value to the caller.

Example:

```
void LogGift(
    decimal amount)
{
    Console.WriteLine(
        $"Gift: {amount:C}");
}
```

Call:

```
LogGift(
    1000m);
```

You cannot write:

```
decimal result =
    LogGift(1000m);
```

because the method returns no value.

---

void Can Still Use return

A `void` method can use:

```
return;
```

to exit early.

Example:

```
void ProcessGift(
    decimal amount)
{
    if (amount <= 0m)
    {
        return;
    }

    Console.WriteLine(
        "Processing gift");
}
```

When Should a Method Return a Value?

Return a value when the caller needs information produced by the method.

Good:

```
decimal CalculateGiftTotal(...)
```

because calculation naturally produces a value.

Also:

```
bool IsEligibleForMatching(...)
```

because the caller needs a yes/no answer.

When Should a Method Be void?

Use `void` when the purpose is to perform an action and the caller does not need a direct result.

Example:

```
void WriteAuditMessage(...)
```

But in real application architecture, many operations benefit from returning meaningful information such as:

```
Result
Status
Created object
Validation result
```

instead of hiding all outcomes inside `void` methods.

Pass by Value — Default Behavior

This is one of the most important concepts in this topic.

By default, C# passes arguments **by value**.

That means the method receives a copy of the argument variable's value.

But what gets copied depends on whether the type is:

```
Value Type
or
Reference Type
```

Value Type Passed by Value

Example:

```
void ChangeAmount(
    decimal amount)
{
    amount = 5000m;
}
```

Caller:

```
decimal giftAmount = 1000m;

ChangeAmount(
    giftAmount);

Console.WriteLine(
    giftAmount);
```

Output:

```
1000
```

Why?

Because:

```
giftAmount = 1000
    →
Copy passed to method
    →
amount = 1000
    →
method changes copy to 5000
    →
original remains 1000
```

Value Type Pass-by-Value Flow

```
Caller

giftAmount
1000
    →
Copy
    →
Method Parameter

amount
1000
    →
change to 5000

Caller giftAmount
still 1000
```

Reference Type Passed by Value

This causes frequent confusion.

Suppose:

```
public class Gift
{
    public decimal Amount { get; set; }
}
```

Method:

```
void ChangeGift(
    Gift gift)
{
    gift.Amount = 5000m;
}
```

Caller:

```
Gift gift =
    new Gift
    {
        Amount = 1000m
    };

ChangeGift(
    gift);

Console.WriteLine(
    gift.Amount);
```

Output:

```
5000
```

Why?

Because when a reference type is passed normally, C# passes a **copy of the reference**, not a copy of the entire object.

Both references point to the same object.

Reference Type Pass-by-Value Flow

```
Caller Variable
gift
    →
Reference A
    →
Gift Object
Amount = 1000


Method receives
copy of Reference A
    →
also points to
same Gift Object
```

Method changes:

```
Gift Object.Amount
    →
5000
```

Caller sees:

```
5000
```

Reassigning Reference Parameter

Now look at this:

```
void ReplaceGift(
    Gift gift)
{
    gift =
        new Gift
        {
            Amount = 9000m
        };
}
```

Caller:

```
Gift gift =
    new Gift
    {
        Amount = 1000m
    };

ReplaceGift(
    gift);

Console.WriteLine(
    gift.Amount);
```

Result:

```
1000
```

Why?

The method received a copy of the reference.

It changed its local copy to point to another object.

The caller's reference still points to the original object.

Important Pass-by-Value Rule

For normal parameters:

```
Value Type
    →
Copy of value

Reference Type
    →
Copy of reference
```

This distinction is one of the most important C# interview concepts.

---

ref Parameter

Definition

`ref` passes the caller's variable **by reference**, allowing the method to work with the caller's actual variable storage.

The caller must initialize the variable before passing it.

Both declaration and call normally include:

```
ref
```

Microsoft documents `ref` as a reference parameter that can be read and reassigned by the called method.

---

ref Example

```
void IncreaseAmount(
    ref decimal amount)
{
    amount += 100m;
}
```

Caller:

```
decimal giftAmount = 1000m;

IncreaseAmount(
    ref giftAmount);

Console.WriteLine(
    giftAmount);
```

Result:

```
1100
```

ref Flow

```
Caller

giftAmount
1000
    →
Pass actual variable by reference
    →
Method parameter amount
    →
amount += 100
    →
Caller variable becomes 1100
```

No separate independent variable value is being changed here.

ref Requires Initialization

This is invalid:

```
decimal amount;

// IncreaseAmount(
//     ref amount);
```

because a `ref` argument must already have a value before the call.

Correct:

```
decimal amount = 0m;

IncreaseAmount(
    ref amount);
```

When to Use ref

Possible use cases include:

```
Method needs to modify caller variable

Performance-sensitive large struct scenarios

Low-level APIs

Interop
```

But do not use `ref` casually.

Returning a value is usually clearer.

Instead of:

```
void CalculateTotal(
    ref decimal total)
```

prefer:

```
decimal CalculateTotal(...)
```

when a normal return value expresses the operation naturally.

out Parameter

Definition

An `out` parameter is used when a method must assign a value and return it through a parameter.

The caller does **not** need to initialize the variable first.

The method **must** assign it before returning.

out Example

```
void GetGiftSummary(
    out int giftCount,
    out decimal totalAmount)
{
    giftCount = 10;
    totalAmount = 5000m;
}
```

Call:

```
GetGiftSummary(
    out int giftCount,
    out decimal totalAmount);

Console.WriteLine(
    giftCount);

Console.WriteLine(
    totalAmount);
```

out Flow

```
Caller
    →
No initial value required
    →
Method receives output slot
    →
Method MUST assign
    →
Returns
    →
Caller receives assigned value
```

Classic TryParse Example

You already used:

```
if (decimal.TryParse(
        amountText,
        out decimal amount))
{
}
```

`TryParse`:

```
Returns bool
    →
Success / Failure

out parameter
    →
Parsed value
```

This is a classic example of an `out` parameter.

Why TryParse Uses out

Method needs to return two pieces of information:

```
Did parsing succeed?
    →
bool

What was parsed?
    →
decimal
```

So:

```
bool success =
    decimal.TryParse(
        text,
        out decimal amount);
```

---

out Variable Declaration at Call Site

Modern C# allows:

```
decimal.TryParse(
    text,
    out decimal amount);
```

instead of declaring first:

```
decimal amount;

decimal.TryParse(
    text,
    out amount);
```

The inline version is cleaner and common.

---

out Must Be Assigned

This is invalid:

```
void GetAmount(
    out decimal amount)
{
    // No assignment.
}
```

The compiler requires:

```
amount = ...;
```

on every path before the method returns.

---

ref vs out

ref	out
Variable must already be initialized	Initialization not required
Method may read existing value	Existing value is not the purpose
Method may modify/reassign	Method must assign
Caller writes `ref`	Caller writes `out`

in Parameter

Definition

An `in` parameter passes an argument by reference but does not allow the method to assign a new value to the parameter.

It is conceptually:

```
Reference
+
Read Only
```

Microsoft describes `in` as a read-only reference parameter and notes it can help avoid copying larger value types.

in Example

Suppose:

```
public readonly struct GiftSummary
{
    public decimal Total { get; init; }
    public int Count { get; init; }
}
```

Method:

```
void DisplaySummary(
    in GiftSummary summary)
{
    Console.WriteLine(
        summary.Total);

    // Not allowed:
    // summary = new GiftSummary();
}
```

in Flow

```
Caller Variable
    →
Passed by reference
    →
Method can read
    →
Method cannot reassign parameter
```

Why in Exists

For small types such as:

```
int
bool
decimal
```

ordinary value passing is usually perfectly fine.

But a large struct might be expensive to copy repeatedly.

`in` can allow:

```
No full struct copy
+
No modification through parameter
```

in appropriate scenarios.

Do Not Overuse in

Do not write:

```
void Process(
    in int value)
```

everywhere simply because `in` exists.

For small values, ordinary parameter passing is simpler.

Use `in` when:

```
Large value type
+
Measured or reasonable copy concern
+
Read-only behavior desired
```

---

ref readonly Parameter

Modern C# also supports:

```
ref readonly
```

parameters.

They represent another read-only-by-reference form.

Microsoft's current parameter documentation distinguishes `in` from `ref readonly`; both prevent assignment through the parameter, but their calling and binding rules differ.

This is advanced C#.

For most business application development:

```
normal parameter
ref
out
in
```

are the first concepts to master.

Parameter Modifier Comparison

Modifier	Caller must initialize?	Method can read existing value?	Method can assign?
Normal	Yes	Yes	Local parameter only
`ref`	Yes	Yes	Yes, caller variable changes
`out`	No	Not required as input	Must assign
`in`	Yes	Yes	No
`ref readonly`	Yes	Yes	No

Important async Restriction

Methods declared:

```
async
```

cannot use `ref`, `out`, or `in` reference parameters in the normal way.

Microsoft's parameter documentation also states that these reference parameter modifiers aren't supported on async methods or iterator methods.

Why?

Because async methods can suspend and resume later, while reference parameters refer directly to caller storage whose lifetime rules do not fit that model safely.

Later, when we learn async/await, this becomes important.

Optional Parameters

Definition

An **optional parameter** has a default value.

The caller may omit that argument.

Example:

```
decimal CalculateFee(
    decimal amount,
    decimal feeRate = 0.02m)
{
    return amount * feeRate;
}
```

Call:

```
decimal fee =
    CalculateFee(
        1000m);
```

C# uses:

```
feeRate = 0.02
```

Result:

```
20
```

Passing Optional Argument Explicitly

```
decimal fee =
    CalculateFee(
        1000m,
        0.03m);
```

Now:

```
feeRate
    →
0.03
```

Result:

```
30
```

Optional Parameter Flow

```
Method Call
    →
Argument supplied?
    →
Yes
    →
Use supplied value

No
    →
Use default value
```

Optional Parameter Rule

Required parameters must generally appear before optional parameters in the declaration.

Good:

```
void ProcessGift(
    decimal amount,
    bool sendReceipt = true)
```

Bad concept:

```
// optional first, required later
```

because callers need a clear parameter sequence.

Optional Parameter Defaults

Default parameter values must be compile-time-compatible default expressions allowed by the language rules.

Typical:

```
void LogGift(
    string message = "Gift processed")
```

or:

```
void Process(
    int retryCount = 3)
```

Be Careful with Changing Defaults

Optional parameter values can be baked into compiled caller code.

Therefore changing a library's default parameter value may not affect already compiled callers until they are recompiled.

This matters more in shared libraries and versioned APIs.

For internal application code, it is still useful to understand.

Named Arguments

Normally arguments are positional.

Example:

```
CreateGift(
    1001,
    5000m,
    false);
```

What does `false` mean?

Without seeing the method declaration, this is unclear.

Named arguments improve readability.

Example:

```
CreateGift(
    constituentId: 1001,
    totalAmount: 5000m,
    isAnonymous: false);
```

Now the meaning is obvious.

Positional Arguments

Example:

```
CalculateRemaining(
    1000m,
    700m);
```

Arguments map based on position:

```
1000m
    →
first parameter

700m
    →
second parameter
```

Named Arguments Can Change Ordering

Suppose:

```
void CreateGift(
    long constituentId,
    decimal amount,
    bool isAnonymous)
```

You can write:

```
CreateGift(
    amount: 1000m,
    isAnonymous: false,
    constituentId: 5001);
```

Because the names tell C# which parameter receives each value.

When Named Arguments Are Useful

Useful when:

```
Many parameters

Several Boolean parameters

Optional parameters

Meaning isn't obvious

Call site readability matters
```

Especially bad:

```
ConfigureGift(
    true,
    false,
    true,
    false);
```

Better:

```
ConfigureGift(
    sendReceipt: true,
    allowRefund: false,
    notifyFundraiser: true,
    isAnonymous: false);
```

Though many Boolean parameters may itself indicate the method needs redesign.

Named Arguments and API Design

If a method needs:

```
8
10
15
```

parameters, named arguments improve readability but do not solve the deeper design problem.

You might need:

```
Request object
Options object
Value object
Domain object
```

instead.

Example:

```
CreateGiftRequest request
```

is often better than 12 independent parameters.

---

params Parameter

Definition

`params` allows callers to provide a variable number of arguments.

Classic form:

```
decimal CalculateTotal(
    params decimal[] amounts)
{
    decimal total = 0m;

    foreach (decimal amount
             in amounts)
    {
        total += amount;
    }

    return total;
}
```

Call:

```
decimal total =
    CalculateTotal(
        100m,
        200m,
        300m);
```

Result:

```
600
```

Calling params with No Arguments

```
decimal total =
    CalculateTotal();
```

The parameter collection is empty.

Result:

```
0
```

Calling params with an Array

```
decimal[] gifts =
{
    100m,
    200m,
    300m
};

decimal total =
    CalculateTotal(
        gifts);
```

also works.

params Rule

A method can have only one `params` parameter, and it must be the last parameter in the method declaration.

Good:

```
void Log(
    string category,
    params string[] messages)
```

Modern params in Current C#

Historically, `params` was primarily associated with one-dimensional arrays.

Modern C# supports broader collection forms for `params`, including supported spans and collection types under the current language rules.

For beginner and ordinary application code:

```
params T[]
```

remains the clearest form to learn first.

params Example — Gift Allocations

```
decimal CalculateAllocationTotal(
    params decimal[] allocations)
{
    decimal total = 0m;

    foreach (decimal allocation
             in allocations)
    {
        total += allocation;
    }

    return total;
}
```

Call:

```
decimal allocated =
    CalculateAllocationTotal(
        500m,
        250m,
        250m);
```

Result:

```
1000
```

Method Overloading

Definition

Method overloading allows several methods to have the same name as long as their signatures differ appropriately.

Example:

```
decimal CalculateFee(
    decimal amount)
{
    return amount * 0.02m;
}
```

and:

```
decimal CalculateFee(
    decimal amount,
    decimal rate)
{
    return amount * rate;
}
```

Call:

```
CalculateFee(
    1000m);
```

uses first overload.

Call:

```
CalculateFee(
    1000m,
    0.03m);
```

uses second.

Microsoft describes method overloading as multiple methods with the same name but unique signatures.

Why Overloading Exists

Overloads let one conceptual operation support different input forms.

Example:

```
FindGift(giftId)

FindGift(referenceNumber)

FindGift(tenantId, giftId)
```

Same conceptual operation:

```
Find Gift
```

Different input requirements.

Overload by Number of Parameters

```
void ProcessGift(
    decimal amount)
{
}
```

```
void ProcessGift(
    decimal amount,
    bool sendReceipt)
{
}
```

These are valid overloads.

Overload by Parameter Types

```
void FindGift(
    long giftId)
{
}
```

```
void FindGift(
    string referenceNumber)
{
}
```

Also valid.

Return Type Alone Cannot Overload

This is invalid:

```
int GetValue()
{
    return 1;
}
```

and:

```
string GetValue()
{
    return "1";
}
```

The methods differ only by return type.

C# does not consider return type alone sufficient for method overloading.

Why?

Call:

```
GetValue();
```

How should the compiler know which method you mean?

Method Signature

For overloading purposes, a method's signature is based largely on:

```
Method Name
+
Type Parameter Count
+
Parameter Types
+
Parameter Passing Modes
```

Return type is not used to distinguish overloads.

Overloading and ref/out/in

Reference parameter modes participate in signatures, but C# has rules preventing certain overloads that differ only among some `ref`, `out`, and `in` combinations.

This is an advanced language rule.

For business code, avoid designing overload sets that differ only by subtle parameter modifiers.

They are difficult to understand.

Bad Overloading

Avoid overloads that make caller intent ambiguous or confusing.

Example:

```
ProcessGift(int)

ProcessGift(long)

ProcessGift(decimal)
```

could produce surprising overload-resolution behavior with literals or conversions.

Overload when the variants represent one clear operation.

Expression-Bodied Methods

For simple methods, C# supports:

```
=>
```

syntax.

Instead of:

```
decimal CalculateFee(
    decimal amount)
{
    return amount * 0.02m;
}
```

you can write:

```
decimal CalculateFee(
    decimal amount)
    =>
    amount * 0.02m;
```

Microsoft documents expression-bodied methods as a concise form for methods that return an expression or consist of one statement.

Expression-Bodied void Method

```
void LogGift(
    decimal amount)
    =>
    Console.WriteLine(
        amount);
```

When Expression-Bodied Methods Are Good

Good:

```
bool IsValidGift(
    decimal amount)
    =>
    amount > 0m;
```

Clear and short.

Avoid:

```
Large complex expression
Multiple nested conditional operators
Many chained calls
```

just to keep the method on one line.

Readability is more important.

Static Methods

Definition

A `static` method belongs to the **type itself**, not to a particular object instance.

Example:

```
public static decimal CalculateFee(
    decimal amount)
{
    return amount * 0.02m;
}
```

Call:

```
decimal fee =
    GiftCalculator.CalculateFee(
        1000m);
```

No `GiftCalculator` object is required.

Static Method Flow

```
Type
GiftCalculator
    →
Static Method
CalculateFee
    →
Call directly using type
```

When Static Methods Are Useful

Good for logic that:

```
Does not require object state

Is naturally utility-like

Depends only on supplied inputs

Represents pure calculations
```

Example:

```
public static decimal CalculatePercentage(
    decimal value,
    decimal total)
```

Static Method Limitation

A static method cannot directly access instance fields/properties because there is no specific object instance.

Example:

```
public class Gift
{
    public decimal Amount { get; set; }

    public static void PrintAmount()
    {
        // Cannot directly use Amount here
        // because there is no particular Gift instance.
    }
}
```

Instance Methods

Definition

An **instance method belongs to a specific object instance**.

Example:

```
public class Gift
{
    public decimal Amount { get; private set; }

    public void ChangeAmount(
        decimal newAmount)
    {
        Amount = newAmount;
    }
}
```

Call:

```
Gift gift =
    new Gift();

gift.ChangeAmount(
    1000m);
```

Instance Method Flow

```
Gift Object
    →
Has State
Amount
    →
Call ChangeAmount
    →
Object state changes
```

Static vs Instance Methods

Static Method	Instance Method
Belongs to type	Belongs to object
No object required	Requires object
Cannot directly use instance state	Can use instance state
Good for stateless operations	Good for object/domain behavior

Fundraising Example

Static utility:

```
public static decimal CalculateFee(
    decimal amount,
    decimal rate)
{
    return amount * rate;
}
```

Instance domain behavior:

```
public void Allocate(
    decimal amount)
{
    // changes this Gift's state
}
```

The second logically belongs to a specific gift.

Returning Multiple Values

Sometimes a method logically produces more than one result.

Possible approaches include:

```
Tuple

Custom object/record

out parameters
```

Returning Tuple

Example:

```
(int GiftCount, decimal TotalAmount)
    CalculateSummary(
        decimal[] gifts)
{
    int count = 0;
    decimal total = 0m;

    foreach (decimal gift in gifts)
    {
        if (gift <= 0m)
        {
            continue;
        }

        count++;
        total += gift;
    }

    return (
        count,
        total);
}
```

Call:

```
var result =
    CalculateSummary(
        gifts);

Console.WriteLine(
    result.GiftCount);

Console.WriteLine(
    result.TotalAmount);
```

Tuple Deconstruction

You can write:

```
(
    int giftCount,
    decimal totalAmount
) =
    CalculateSummary(
        gifts);
```

Then:

```
Console.WriteLine(
    giftCount);
```

and:

```
Console.WriteLine(
    totalAmount);
```

Tuple Flow

```
Method
    →
Calculates
    →
Count + Total
    →
Returns Tuple
    →
Caller deconstructs
```

Tuple vs out

Instead of:

```
void GetSummary(
    out int count,
    out decimal total)
```

you can often return:

```
(int Count, decimal Total)
    GetSummary()
```

Tuple returns often communicate method output more naturally.

When Not to Use Tuple

If the result represents an important business concept, create a dedicated type.

Instead of:

```
(int Count,
 decimal Total,
 decimal Average,
 decimal Largest)
```

consider:

```
public sealed record GiftSummary(
    int Count,
    decimal Total,
    decimal Average,
    decimal Largest);
```

Why?

A named domain type provides:

```
Meaning

Validation

Extensibility

Documentation

Reuse
```

Tuples are best for small local groupings of values.

Local Functions

Definition

A **local function is a method-like function declared inside another member**.

Example:

```
decimal CalculateGiftTotal(
    decimal[] gifts)
{
    bool IsValid(
        decimal amount)
    {
        return amount > 0m;
    }

    decimal total = 0m;

    foreach (decimal gift
             in gifts)
    {
        if (IsValid(gift))
        {
            total += gift;
        }
    }

    return total;
}
```

`IsValid` is accessible only within:

```
CalculateGiftTotal
```

Microsoft recommends local functions when a helper benefits from a name, has several statements, needs recursion, or is used more than once within the containing member.

Why Local Functions Exist

Sometimes helper logic:

```
belongs only to one method
```

Creating a private class-level method may expose it to more of the class than necessary.

A local function keeps it close to where it is used.

Local Function Scope

```
Containing Method
    →
Local Function
    →
Available only inside containing member
```

Local Function Capturing Variables

Local functions can access variables from the containing method.

Example:

```
decimal CalculateTotal(
    decimal[] gifts,
    decimal minimumAmount)
{
    bool IsIncluded(
        decimal amount)
    {
        return amount >= minimumAmount;
    }

    decimal total = 0m;

    foreach (decimal gift in gifts)
    {
        if (IsIncluded(gift))
        {
            total += gift;
        }
    }

    return total;
}
```

The local function accesses:

```
minimumAmount
```

from the outer method.

Local Function vs Separate Private Method

Use local function when:

```
Helper is meaningful only inside one method
```

Use private method when:

```
Several methods need it

It represents a clear class responsibility

It deserves independent testing/reuse
```

Local Function vs Lambda

Both can represent local behavior.

Local function:

```
bool IsValid(
    decimal amount)
{
    return amount > 0m;
}
```

Lambda:

```
Func<decimal, bool> isValid =
    amount => amount > 0m;
```

Use local function when you want:

```
Named helper

Recursion

Several statements

Clear method-like structure
```

Lambdas are especially useful with:

```
LINQ

Delegates

Callbacks
```

and will be studied later.

Recursion

Definition

Recursion occurs when a method calls itself.

Example:

```
int Sum(
    int number)
{
    if (number <= 1)
    {
        return number;
    }

    return number
           + Sum(
               number - 1);
}
```

Call:

```
int result =
    Sum(5);
```

Flow:

```
Sum(5)
    →
5 + Sum(4)
    →
5 + 4 + Sum(3)
    →
5 + 4 + 3 + Sum(2)
    →
5 + 4 + 3 + 2 + Sum(1)
    →
15
```

Base Case

Every recursive method needs a stopping condition called the:

```
Base Case
```

Example:

```
if (number <= 1)
{
    return number;
}
```

Without it:

```
Method calls itself
    →
again
    →
again
    →
again
```

until stack exhaustion.

Microsoft explicitly notes that recursive functions need a condition that stops recursion.

Infinite Recursion

Bad:

```
void Process()
{
    Process();
}
```

This never stops.

Eventually:

```
Stack Overflow
```

can occur.

When Recursion Is Useful

Good candidates include:

```
Tree structures

Hierarchical data

Graph traversal

File/folder structures

Recursive mathematical definitions

Parser structures
```

For your Fundraising Platform, if you later had a hierarchical fund or campaign structure:

```
Parent Campaign
    →
Child Campaign
        →
Child Campaign
```

recursion could potentially process that hierarchy.

When Not to Use Recursion

If a simple loop expresses the logic more clearly:

```
Use loop
```

Microsoft's local-function guidance similarly recommends recursion when recursive structure makes the solution easier to understand, and a loop when repeated work is more naturally iterative.

Method Parameters Are Local Variables

Method parameters exist inside the method's scope.

Example:

```
void ProcessGift(
    decimal amount)
{
    Console.WriteLine(
        amount);
}
```

`amount` exists only inside:

```
ProcessGift
```

You cannot access it from unrelated code outside that method.

Parameter Scope

```
Caller
    →
Argument

Method starts
    →
Parameter created/bound

Method ends
    →
Parameter scope ends
```

Local Variables Inside Methods

Example:

```
decimal CalculateNet(
    decimal amount,
    decimal fee)
{
    decimal net =
        amount - fee;

    return net;
}
```

`net` exists only while the method executes.

Method Call Stack Concept

When one method calls another:

```
ProcessGift()
    →
ValidateGift()
    →
CalculateFee()
```

runtime execution conceptually forms call frames.

```
ProcessGift
    →
calls ValidateGift
        →
        ValidateGift returns
    →
calls CalculateFee
        →
        CalculateFee returns
    →
ProcessGift continues
```

Each call has its own local parameters and local variables.

This becomes important when studying:

```
Stack
Recursion
Exceptions
Debugging Call Stack
```

later.

Method Design — One Clear Responsibility

A method should ideally have one clear purpose.

Bad:

```
void ProcessGift()
{
    ValidateGift();
    SaveGiftToDatabase();
    SendEmail();
    GeneratePdf();
    UploadDocument();
    UpdateAnalytics();
    CallPaymentProvider();
    WriteAuditFile();
}
```

This method is doing too much.

Later SOLID principles will call this a responsibility-design problem.

A better architecture separates operations.

Good Method Names

Method names should describe actions.

Examples:

```
CalculateGiftTotal

ValidateAllocation

CreateGift

SendReceipt

GetConstituent

IsCampaignActive

CanRefundGift
```

Avoid vague names:

```
DoStuff

HandleData

ProcessThing

RunLogic
```

unless the surrounding abstraction gives them clear meaning.

Boolean Method Naming

Methods returning `bool` often read naturally as questions:

```
bool IsValidGift(...)
```

```
bool HasPermission(...)
```

```
bool CanRefund(...)
```

```
bool ShouldSendReceipt(...)
```

Then callers read clearly:

```
if (CanRefund(gift))
{
}
```

Methods Should Not Have Too Many Parameters

This is a warning sign:

```
CreateGift(
    tenantId,
    constituentId,
    campaignId,
    fundId,
    amount,
    currency,
    date,
    source,
    isAnonymous,
    notes,
    referenceNumber,
    userId,
    sendReceipt);
```

Even with named arguments, this is difficult to manage.

Better:

```
CreateGiftRequest request
```

or:

```
Gift gift
```

depending on the architecture.

Parameter Object Pattern Preview

Instead of:

```
void CreateGift(
    long tenantId,
    long constituentId,
    long? campaignId,
    decimal amount,
    string currencyCode,
    bool isAnonymous)
```

you might later define:

```
public sealed record CreateGiftRequest(
    long TenantId,
    long ConstituentId,
    long? CampaignId,
    decimal Amount,
    string CurrencyCode,
    bool IsAnonymous);
```

Then:

```
void CreateGift(
    CreateGiftRequest request)
```

This becomes especially useful in:

```
ASP.NET Core Request DTOs
Application Commands
Service Methods
```

Pure Methods

A **pure method** generally:

```
Depends only on inputs

Does not modify external state

Produces the same result for same inputs
```

Example:

```
decimal CalculateFee(
    decimal amount,
    decimal rate)
{
    return amount * rate;
}
```

For:

```
amount = 1000
rate = 0.02
```

result is always:

```
20
```

Pure methods are generally:

```
Easy to test

Easy to understand

Easy to reuse
```

Method with Side Effects

Example:

```
void SendReceipt(
    Gift gift)
{
    emailService.Send(...);
}
```

This performs an external action.

Possible side effects include:

```
Database updates

Email sending

File changes

Network calls

Object mutation

Logging
```

Side effects are sometimes necessary.

The important point is to make them clear.

Fundraising Example — Complete Gift Calculation Methods

```
static bool IsValidGift(
    decimal amount)
{
    return amount > 0m;
}

static decimal CalculateFee(
    decimal amount,
    decimal rate = 0.02m)
{
    return amount * rate;
}

static decimal CalculateNetAmount(
    decimal amount,
    decimal fee)
{
    return amount - fee;
}

static string GetGiftCategory(
    decimal amount)
{
    return amount switch
    {
        >= 10_000m
            => "Principal Gift",

        >= 5_000m
            => "Major Gift",

        >= 1_000m
            => "Leadership Gift",

        > 0m
            => "Standard Gift",

        _
            => "Invalid Gift"
    };
}
```

Usage:

```
decimal giftAmount =
    7500m;

if (!IsValidGift(
        giftAmount))
{
    return;
}

decimal fee =
    CalculateFee(
        giftAmount);

decimal netAmount =
    CalculateNetAmount(
        giftAmount,
        fee);

string category =
    GetGiftCategory(
        giftAmount);
```

Flow:

```
Gift Amount
    →
IsValidGift
    →
Valid?
    →
CalculateFee
    →
CalculateNetAmount
    →
GetGiftCategory
    →
Continue Processing
```

Fundraising Example — Validate Allocation

```
static bool IsFullyAllocated(
    decimal giftAmount,
    params decimal[] allocations)
{
    decimal allocatedTotal = 0m;

    foreach (decimal allocation
             in allocations)
    {
        if (allocation <= 0m)
        {
            return false;
        }

        allocatedTotal += allocation;
    }

    return allocatedTotal
           == giftAmount;
}
```

Call:

```
bool valid =
    IsFullyAllocated(
        1000m,
        700m,
        300m);
```

Result:

```
true
```

Flow:

```
Gift = 1000
    →
Allocations
700 + 300
    →
Each > 0?
    →
Yes
    →
Total = 1000?
    →
Yes
    →
Valid
```

Fundraising Example — Multiple Outputs

```
static (
    int Count,
    decimal Total,
    decimal Average)
    CalculateGiftSummary(
        decimal[] gifts)
{
    int count = 0;
    decimal total = 0m;

    foreach (decimal gift
             in gifts)
    {
        if (gift <= 0m)
        {
            continue;
        }

        count++;
        total += gift;
    }

    decimal average =
        count == 0
            ? 0m
            : total / count;

    return (
        count,
        total,
        average);
}
```

Call:

```
var summary =
    CalculateGiftSummary(
        new[]
        {
            100m,
            200m,
            300m
        });
```

Result:

```
Count   = 3
Total   = 600
Average = 200
```

Common Mistake — Confusing Argument and Parameter

Incorrect terminology:

> "1000 is the parameter."

More accurately:

```
Method declaration
    →
parameter

Method call
    →
argument
```

Common Mistake — Expecting Normal Value Parameter to Modify Caller

```
void Change(
    int value)
{
    value = 100;
}
```

Caller:

```
int number = 10;

Change(number);
```

`number` remains:

```
10
```

because normal parameters are passed by value.

Common Mistake — Believing Reference Types Are Passed by Reference Automatically

This statement is incorrect:

> Classes are passed by reference.

More precise:

> A class reference is normally passed **by value**. The copied reference still points to the same object.

That distinction explains why:

```
Mutating object
```

can affect caller-visible state,

while:

```
Reassigning parameter
```

does not replace the caller's variable.

Common Mistake — Using ref to Return One Value

Avoid:

```
void CalculateFee(
    decimal amount,
    ref decimal fee)
{
    fee =
        amount * 0.02m;
}
```

Better:

```
decimal CalculateFee(
    decimal amount)
{
    return amount * 0.02m;
}
```

The return type communicates intent more clearly.

Common Mistake — Too Many out Parameters

This:

```
void GetGiftData(
    out long giftId,
    out decimal amount,
    out string currency,
    out DateOnly date,
    out bool anonymous)
```

is harder to maintain.

A dedicated result type is usually better.

Common Mistake — Overusing Optional Parameters

Avoid creating methods whose behavior changes dramatically based on many optional flags:

```
ProcessGift(
    gift,
    true,
    false,
    true,
    false,
    null,
    3);
```

This is difficult to understand.

Prefer:

```
Dedicated options type

Separate methods

Clear domain operations
```

depending on the situation.

Common Mistake — Deep Recursion Without Need

Recursion uses call-stack frames.

Very deep recursion can eventually produce:

```
StackOverflowException
```

When simple iteration solves the problem clearly, prefer iteration.

Common Mistake — Method Doing Too Much

A method named:

```
ProcessGift
```

should not contain hundreds of lines handling:

```
HTTP
Database
Email
PDF
Audit
Payment
Analytics
Caching
```

This creates high coupling and difficult testing.

Later SOLID and architecture topics will show how to separate these responsibilities.

Common Mistake — Hidden Side Effects

Suppose:

```
decimal CalculateFee(
    Gift gift)
```

sounds like a calculation.

If it unexpectedly:

```
updates database
sends email
changes gift status
```

the name and behavior are misleading.

Methods should make side effects clear.

Performance Considerations

Method calls themselves are normally inexpensive.

Do not avoid methods merely to save call overhead.

The JIT compiler can inline suitable small methods, which can eliminate some method-call overhead automatically.

Focus on:

```
Correct design
Readability
Algorithm
I/O
Database calls
Network calls
Allocations
```

before worrying about tiny method-call cost.

Large Struct Parameters

For large value types, passing by value can copy the struct.

In performance-sensitive code, `in` or `ref readonly` may reduce copying.

But:

```
Measure first.
```

Do not apply reference parameters everywhere.

Avoid Allocations in params Hot Paths

A traditional:

```
params T[]
```

call may require creation of an array when arguments are provided individually.

Example:

```
CalculateTotal(
    1m,
    2m,
    3m);
```

can involve a parameter collection.

For normal business code this is rarely important.

For very hot performance-sensitive code, modern `params` collection forms and alternative APIs may matter.

Again:

```
Measure before optimizing.
```

Security Considerations

Methods define boundaries.

Validate untrusted input at appropriate boundaries.

Example:

```
void ProcessGift(
    decimal amount)
{
    if (amount <= 0m)
    {
        throw new ArgumentOutOfRangeException(
            nameof(amount));
    }
}
```

Do not assume callers always provide valid data.

Do Not Pass Secrets Unnecessarily

Avoid:

```
ProcessGift(
    cardNumber,
    cvv,
    password,
    token,
    secretKey,
    ...)
```

through many layers when not required.

Sensitive information should be:

```
Minimized

Scoped appropriately

Never casually logged

Handled through secure abstractions
```

Method Boundaries and Validation

Suppose a service receives:

```
CreateGift(
    decimal amount)
```

The method should not assume:

```
amount is valid
```

unless the architecture guarantees it through a validated domain type.

Validation can occur:

```
API boundary
Application layer
Domain object
```

depending on the rule.

Later architecture topics will clarify this.

Practical Exercise — Add Two Values

Create:

```
decimal Add(
    decimal first,
    decimal second)
{
    return first + second;
}
```

Call:

```
decimal result =
    Add(
        100m,
        200m);
```

Expected:

```
300
```

Practical Exercise — Validate Gift

Requirement:

> Return true only when gift amount is greater than zero.

```
static bool IsValidGift(
    decimal amount)
{
    return amount > 0m;
}
```

Practical Exercise — Optional Fee Rate

```
static decimal CalculateFee(
    decimal amount,
    decimal rate = 0.02m)
{
    return amount * rate;
}
```

Call:

```
CalculateFee(
    1000m);
```

Result:

```
20
```

Call:

```
CalculateFee(
    1000m,
    0.03m);
```

Result:

```
30
```

Practical Exercise — ref

```
static void AddBonus(
    ref decimal amount)
{
    amount += 100m;
}
```

Caller:

```
decimal amount = 1000m;

AddBonus(
    ref amount);
```

Expected:

```
1100
```

Practical Exercise — out

```
static bool TryCalculatePercentage(
    decimal value,
    decimal total,
    out decimal percentage)
{
    if (total == 0m)
    {
        percentage = 0m;
        return false;
    }

    percentage =
        value / total * 100m;

    return true;
}
```

Call:

```
if (TryCalculatePercentage(
        75m,
        100m,
        out decimal percentage))
{
    Console.WriteLine(
        percentage);
}
```

Expected:

```
75
```

Practical Exercise — params

```
static decimal SumGifts(
    params decimal[] gifts)
{
    decimal total = 0m;

    foreach (decimal gift in gifts)
    {
        total += gift;
    }

    return total;
}
```

Call:

```
decimal total =
    SumGifts(
        100m,
        200m,
        300m,
        400m);
```

Expected:

```
1000
```

Practical Exercise — Tuple

```
static (
    decimal Min,
    decimal Max)
    GetRange(
        decimal[] values)
{
    decimal min =
        values[0];

    decimal max =
        values[0];

    foreach (decimal value in values)
    {
        if (value < min)
        {
            min = value;
        }

        if (value > max)
        {
            max = value;
        }
    }

    return (
        min,
        max);
}
```

Practical Exercise — Recursion

Factorial:

```
static long Factorial(
    int number)
{
    if (number <= 1)
    {
        return 1;
    }

    return number
           * Factorial(
               number - 1);
}
```

Call:

```
long result =
    Factorial(5);
```

Flow:

```
5!
    →
5 × 4 × 3 × 2 × 1
    →
120
```

Interview Questions

What is a method in C#?

A method is a named block of code that performs an operation.

It may receive parameters and may return a result.

Example:

```
decimal CalculateFee(
    decimal amount)
{
    return amount * 0.02m;
}
```

What is the difference between a parameter and an argument?

A **parameter** is declared by the method.

An **argument** is the actual value supplied when calling the method.

```
Method declaration
    →
parameter

Method invocation
    →
argument
```

What is a return type?

The return type defines the type of value the method returns.

Example:

```
decimal Calculate(...)
```

returns a `decimal`.

What does void mean?

`void` means the method does not return a value to the caller.

What does return do?

`return` exits the current method.

In a value-returning method it also supplies the result.

Are parameters passed by value or by reference by default?

By default, arguments are passed **by value**.

For value types, the value is copied.

For reference types, the reference itself is copied.

If classes are reference types, are they automatically passed by reference?

No.

A reference-type variable is normally passed **by value**, meaning the reference is copied.

The caller and method then hold references to the same object.

Why can a method modify a class object's properties even though the parameter was passed by value?

Because the copied reference still points to the same object.

Changing the object's state is visible to all references pointing to that object.

Why doesn't reassigning the parameter replace the caller's class variable?

Because the method has only its own copy of the reference.

Changing that copy does not change the caller's variable.

Use `ref` if the method genuinely needs to reassign the caller's variable itself.

What is ref?

`ref` passes a variable by reference.

The variable must be initialized before the call, and the method can read or modify it.

What is out?

`out` passes an output variable by reference.

The caller does not need to initialize it first, but the method must assign it before returning.

What is in?

`in` passes an argument using read-only reference semantics.

The method can read it but cannot reassign through that parameter. It can be useful for avoiding copies of larger value types.

What is the difference between ref, out, and in?

```
ref
    →
Input + output
Caller initializes
Method may modify

out
    →
Output
Caller need not initialize
Method must assign

in
    →
Read-only input by reference
Method cannot assign
```

Can ref, out, or in be used in async methods?

Normal reference parameters of these forms aren't supported for async methods because their lifetime model conflicts with async suspension/resumption. Microsoft documents this restriction for async and iterator methods.

What is an optional parameter?

An optional parameter has a default value.

Example:

```
void Process(
    bool sendReceipt = true)
```

The caller can omit the argument.

What is a named argument?

A named argument specifies the target parameter by name.

Example:

```
CreateGift(
    amount: 1000m,
    isAnonymous: false);
```

This improves readability.

What is params?

`params` allows a method to accept a variable number of arguments.

Example:

```
decimal Sum(
    params decimal[] values)
```

The `params` parameter must be last and only one can exist in a method declaration.

What is method overloading?

Method overloading means declaring multiple methods with the same name but different valid signatures.

Example:

```
ProcessGift(decimal amount)
```

and:

```
ProcessGift(
    decimal amount,
    bool sendReceipt)
```

Can methods be overloaded only by changing the return type?

No.

Return type alone isn't part of overload resolution in a way that allows two otherwise identical method signatures.

What is a method signature?

For normal overloading purposes, method identity is determined by its name and parameter/type-related signature information rather than the return type.

What is an expression-bodied method?

It is a concise method syntax using:

```
=>
```

Example:

```
bool IsValid(
    decimal amount)
    =>
    amount > 0m;
```

What is a local function?

A local function is declared inside another member and is available within that containing member.

It is useful for private helper logic, recursion, or multi-statement local behavior.

What is recursion?

Recursion occurs when a method calls itself.

Every recursive algorithm needs a base case that eventually stops the recursive calls.

What is the difference between static and instance methods?

Static methods belong to the type.

Instance methods belong to a particular object and can work directly with that object's state.

How can a method return multiple values?

Common options include:

```
Tuple

Custom object/record

out parameters
```

For meaningful business results, a dedicated type is often the clearest design.

Best Practices

Give methods clear action-oriented names.

Keep methods focused on one understandable responsibility.

Prefer normal return values over `ref` or `out` when a single result naturally exists.

Use `Try...` methods with `out` when failure is expected and should not be exceptional.

Use `in` only when its read-only by-reference semantics provide real value.

Use optional parameters for genuinely optional behavior, not as a substitute for good API design.

Use named arguments when they improve call-site clarity.

Avoid methods with many Boolean parameters.

Avoid methods with very large parameter lists.

Use a request/options object when many related inputs belong together.

Use overloads only when they represent the same conceptual operation.

Do not overload methods in ways that create ambiguous or surprising calls.

Use expression-bodied methods for genuinely simple methods.

Use local functions for helpers meaningful only inside one member.

Use recursion when the problem is naturally recursive.

Prefer loops when iteration communicates the logic more simply.

Keep calculations pure where practical.

Make side effects clear.

Validate method inputs at the appropriate application/domain boundary.

Quick Revision

Concept	Meaning
Method	Named reusable block of code
Parameter	Input declared by method
Argument	Value supplied by caller
Return type	Type returned by method
`void`	No return value
`return`	Exit method / return result
Normal parameter	Passed by value
`ref`	Read/write caller variable by reference
`out`	Method must produce value through parameter
`in`	Read-only reference parameter
`ref readonly`	Read-only reference form
Optional parameter	Parameter with default value
Named argument	Argument supplied by parameter name
`params`	Variable number of arguments
Overloading	Same method name, different valid signatures
Local function	Function declared inside another member
Expression-bodied method	Concise `=>` method
Static method	Belongs to type
Instance method	Belongs to object
Tuple return	Return multiple values together
Recursion	Method calls itself
Base case	Stops recursion

Complete Fundraising Example

```
static bool IsValidGift(
    decimal amount)
{
    return amount > 0m;
}

static decimal CalculateFee(
    decimal amount,
    decimal feeRate = 0.02m)
{
    return amount * feeRate;
}

static decimal CalculateAllocationTotal(
    params decimal[] allocations)
{
    decimal total = 0m;

    foreach (decimal allocation
             in allocations)
    {
        total += allocation;
    }

    return total;
}

static (
    decimal Fee,
    decimal NetAmount)
    CalculateGiftFinancials(
        decimal giftAmount,
        decimal feeRate = 0.02m)
{
    decimal fee =
        CalculateFee(
            giftAmount,
            feeRate);

    decimal netAmount =
        giftAmount - fee;

    return (
        fee,
        netAmount);
}

decimal giftAmount =
    1000m;

if (!IsValidGift(
        giftAmount))
{
    return;
}

decimal allocated =
    CalculateAllocationTotal(
        700m,
```

```
        300m);

if (allocated != giftAmount)
{
    Console.WriteLine(
        "Gift allocation is invalid.");

    return;
}

var financials =
    CalculateGiftFinancials(
        giftAmount);

Console.WriteLine(
    $"Gift Amount: {giftAmount:C}");

Console.WriteLine(
    $"Fee: {financials.Fee:C}");

Console.WriteLine(
    $"Net: {financials.NetAmount:C}");
```

Business flow:

```
Gift Amount
$1000
    →
IsValidGift
    →
Valid
    →
CalculateAllocationTotal
    →
$700 + $300
    →
$1000
    →
Matches Gift?
    →
Yes
    →
CalculateGiftFinancials
    →
Fee
$20
    →
Net Amount
$980
```

Technical flow:

```
Caller
    →
Arguments
    →
Method Parameters
    →
Method Logic
    →
Return Value
    →
Caller receives result
    →
Next Method
```

Methods are the first major step from simply **writing C# statements** toward **designing software**. Later topics such as OOP, dependency injection, LINQ, asynchronous programming, services, repositories, middleware, and ASP.NET Core controllers all rely heavily on your understanding of method inputs, outputs, state changes, and parameter-passing behavior.

Next roadmap topic: Value Types, Reference Types, Boxing and Unboxing.